

Introduction to Computer Architecture

Sheet #10

Exercises:

1. Suppose we have a computer that uses a memory address word size of 8 bits. This computer has a 16-byte cache and 256 bytes of main memory. The computer accesses a number of memory locations throughout the course of running a program. Suppose this computer uses direct-mapped cache. The format of a memory address as seen by the cache is shown below:

Tag 4 bits	Block 2 bits	Word 2 bits
---------------	-----------------	----------------

The system accesses memory addresses (in hex) in this exact order: 6E, B9, 17, E0, 4E, 4F, 50, 91, A8, A9, AB, AD, 93, and 94. The memory addresses of the first four accesses have been loaded into the cache blocks as shown below. (The contents of the tag are shown in binary in addition to the entire address in hex.)

	Tag Contents	Cache Contents (represented by address)		Tag Contents	Cache Contents (represented by address)
Block 0	1110	E0	Block 1	0001	14
		E1			15
		E2			16
		E3			17
Block 2	1011	B8	Block 3	0110	6C
		B9			6D
		BA			6E
		BB			6F

- a. What is the hit ratio for the entire memory reference sequence given above?
- b. What memory blocks will be in the cache after the last address has been accessed?

ANSWER:

a.

Address	Hit or Miss
6E	Miss, brought into Block 3 with tag 0110 (as shown)
B9	Miss, brought into Block 2 with tag 1011 (as shown)
17	Miss, brought into Block 1 with tag 0001 (as shown)
E0	Miss, brought into Block 0 with tag 1110 (as shown)
4E	Miss, brought into Block 3 with tag 0100
4F	Hit
50	Miss, brought into Block 0 with tag 0101
91	Miss, brought into Block 0 with tag 1001
A8	Miss, brought into Block 2 with tag 1010
A9	Hit
AB	Hit
AD	Miss, brought into Block 3 with tag 1010
93	Hit
94	Hit

Hit ratio is 4 hits out of 14 total accesses, or 28.6% (this is assuming we count the first 4 accesses as misses)

b. Block 0, with tag 1001, contains 90, 91, 92, 93

Block 1, with tag 1001, contains 94, 95, 96, 97

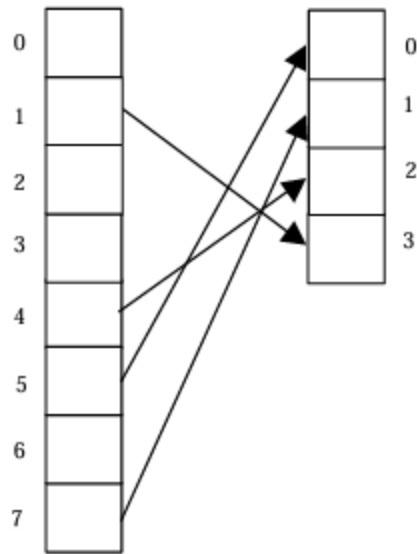
Block 2, with tag 1010, contains A8, A9, AA, AB

Block 3, with tag 1010, contains AC, AD, AE, AF

2. Suppose a process page table contains the entries shown below. Using the format shown in the Figure below, indicate where the process pages are located in memory.

Frame	Valid Bit
--	0
3	1
--	0
--	0
2	1
0	1
--	0
1	1

ANSWER:



3. You have a virtual memory system with a two-entry TLB, a 2-way set associative cache and a page table for a process P. Assume cache blocks of 8 words and page size of 16 words. In the system below, main memory is divided up into blocks, where each block is represented by a letter. Two blocks equals one frame.

0	3
4	1

TLB

Set 0	tag	C	tag	I
Set 1	tag	D	tag	H

Cache

	Frame	Valid
0	3	1
1	0	1
2	-	0
3	2	1
4	1	1
5	-	0
6	-	0
7	-	0

Page Table

Frame	Block
0	C
0	D
1	I
1	J
2	G
2	H
3	A
3	B

Main Memory

Page	Block
0	A
0	B
1	C
1	D
2	E
2	F
3	G
3	H
4	I
4	J
5	K
5	L
6	M
6	N
7	O
7	P

Virtual Memory
For Process P

Given the system state as depicted above, answer the following questions:

- How many bits are in a virtual address for process P? Explain.
- How many bits are in a physical address? Explain.
- Show the address format for virtual address 18_{10} (specify field name and size) that would be used by the system to translate to a physical address and then translate this virtual address into the corresponding physical address. (Hint:

convert 18 to its binary equivalent and divide it into the appropriate fields.) Explain how these fields are used to translate to the corresponding physical address.

- d. Given virtual address 6_{10} converts to physical address 54_{10} . Show the format for a physical address (specify the field names and sizes) that is used to determine the cache location for this address. Explain how to use this format to determine where physical address 54 would be located in cache. (Hint: convert 54 to binary and divide it into the appropriate fields.)

ANSWER:

- a. $2^3 \times 2^4 = 2^7$, so there are 7 bits in a virtual address
- b. $2^2 \times 2^4 = 2^6$, so there are 6 bits in a virtual address
- c. $18 = 001\ 0010$ (where 001 is the page field and 0010 is the offset). Using the page table, and going to entry 1, we see that page 1 maps to frame 0, so the actual physical address would be $00\ 0010$, or 2.
- d. $54 = 11\ 0\ 110$, where 11 is the tag, 0 is the set, and 110 is the offset. Therefore, this would map to Set 0 in cache. Once there, if the tag is found, the block is in cache. If not, it is a miss.

4. Given a virtual memory system with a TLB, a cache, and a page table. Assume the following:
- A TLB hit requires 5ns
 - A cache hit requires 12ns
 - A memory reference requires 25ns
 - A disk reference requires 200ms (this includes updating the page table, cache, and TLB)
 - The TLB hit ratio is 90%
 - The cache hit rate is 98%
 - The page fault rate is .001%
 - On a TLB or cache miss, the time required for access includes a TLB and/or cache update, but the access is NOT restarted
 - On a page fault, the page is fetched from disk, all updates are performed but the access IS restarted
 - All references are sequential (no overlap, nothing done in parallel)

For each of the following, indicate whether or not it is possible. If so, specify the time required for accessing the requested data.

- a. TLB hit, cache hit
- b. TLB miss, page table hit, cache hit
- c. TLB miss, page table hit, cache miss
- d. TLB miss, page table miss, cache hit
- e. TLB miss, page table miss

Write down the equation to calculate the effective access time.

ANSWER:

- a. Possible, 5ns (TLB access) + 12ns (cache access)
- b. Possible, 5ns (TLB access) + 25ns (page table reference) + 12ns (cache access)
- c. Possible, 5ns (TLB access) + 25ns (page table reference) + 12ns (cache access) + 25ns (main memory reference)
- d. Not possible (the cache value would be stale if the page no longer resides in page table)
- e. Possible, 5ns (TLB access) + 25ns (page table reference) + 200ms (disk reference) + 5ns (TLB, since access is restarted) + 12ns (cache hit)

$$\begin{aligned}
 \text{EAT} = & .90[5\text{ns} + \underbrace{.98(12\text{ns})}_{\text{Cache hit}} + \underbrace{.02(25\text{ns})}_{\text{Cache miss}}] \\
 & \underbrace{\hspace{10em}}_{\text{TLB hit}} \\
 & + .10 [5\text{ns} + \underbrace{.998[25\text{ns} + \underbrace{.98(12\text{ns})}_{\text{Cache hit or miss}} + \underbrace{.02(12\text{ns} + 25\text{ns})}_{\text{Page table hit}}]}_{\text{Page table hit}} + \underbrace{.001(25\text{ns} + 200\text{ns} + 5\text{ns})}_{\text{Page table miss}}] \\
 & \underbrace{\hspace{15em}}_{\text{TLB miss}}
 \end{aligned}$$

Restart access, find in TLB

5. A system implements a paged virtual address space for each process using a one-level page table. The maximum size of virtual address space is 16MB. The page table for the running process includes the following valid entries (the $\rightarrow$ notation indicates that a virtual page maps to the given page frame, that is, it is located in that frame):

Virtual page 2 $\rightarrow$ page frame 4	Virtual page 4 $\rightarrow$ page frame 9
Virtual page 1 $\rightarrow$ page frame 2	Virtual page 3 $\rightarrow$ page frame 16
Virtual page 0 $\rightarrow$ page frame 1	

The page size is 1024 bytes and the maximum physical memory size of the machine is 2MB.

- a. How many bits are required for each virtual address?
- b. How many bits are required for each physical address?

- c. What is the maximum number of entries in a page table?
- d. To which physical address will the virtual address 1524_{10} translate?
- e. Which virtual address will translate to physical address 1024_{10} ?

ANSWER:

- a. There are 16MB, or 2^{24} addresses, so we need 24 bits for a virtual address.
- b. Main memory is 2MB, or 2^{21} , so we need 21 bits for a physical address.
- c. There are $2^{24}/2^{10}$ pages in virtual memory, so the page table can have 214 entries.
- d. 1524 is on page 1 (page 0 contains addresses 0=1023, page 1 contains 1024 - 2047), located at offset 500. Page 1 maps to frame 2, so 1524 maps to physical address 2548. You can also find the binary equivalent of 1524 (0000000000001011110100) and divide it into two pieces: the first 14 bits are the page, and the last 10 are the offset. Replace the first 14 bits (00000000000001) by (00000000000010), to get the physical address 00000000000010011110100, or 2548.
- e. Physical address 1024 is at offset 0 in frame 1. Virtual page 0 maps to frame 1, so this is virtual address 0.